# Task W14: Hashing

Hash It Out

### A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

## 1 Overview

This task checks that you understand how hash tables work, not just that you can fill one in. There are four required problems, all done by hand and all about the size of an exam question.

- **Problem 1, Trace It:** build a small table four ways, then remove a key and see what that does to later searches.
- **Problem 2, Find the Mistakes:** check a classmate's work and correct it.
- **Problem 3, Predict and Justify:** say what will happen before it happens, and explain why.
- **Problem 4, Use the Formulas Backward:** start from a performance goal and work out the table size you need.

You will not have Excel or a calculator program on the exam, so work these by hand. A basic calculator is fine for Problem 4.

## 2 Submitting Your Work

### If you find something that needs to be debugged

If, while working through this assignment, you discover something in the problem statements that needs to be debugged, describe it in the space at the end of the answer sheet. This is separate from, and in addition to, your answers.

### Where to submit

Type your answers in the answer sheet that comes with this task (a Word or OpenDocument file), and save it as .docx or .odt. Put it in a folder named `taskw14` in your GitHub repository, and commit it.

## 3 Conventions

These match the lecture and the notes.

- The hash function is $h(x) = x \bmod m$, where $m$ is the table size. Every slot number below is taken mod $m$.
- **Linear probing:** probe $i$ checks slot $h(x) + i$.
- **Quadratic probing:** probe $i$ checks slot $h(x) + i^2$.
- **Double hashing:** probe $i$ checks slot $h(x) + i \cdot h_2(x)$, where $h_2(x) = 7 - (x \bmod 7)$.
- **Separate chaining:** each key goes at the **end** of chain $h(x)$.
- **Counting probes:** every slot you check counts as one probe, including the first. For a search in an open-addressing table, the search stops at the key or at the first empty slot.
- **A key that cannot be placed:** if every slot its probe sequence can reach is full, leave it out of the table, write *not placed*, and go on to the next key.
- **Deleted slots:** a removed key's slot is marked DEL, not left empty.
- **Load factor:** $\alpha$ is the number of keys actually in the table divided by the table size.

## 4 Problem 1: Trace It

Insert these nine keys, in this order, into a table of size $m = 13$:

$$93,\ \ 61,\ \ 51,\ \ 37,\ \ 60,\ \ 68,\ \ 38,\ \ 90,\ \ 89$$

(a) Build the final table four times: with linear probing, with quadratic probing, with double hashing, and with separate chaining. For each of the three open-addressing tables, also give the number of probes each insert used (write *not placed* where a key could not be placed).

(b) Give the load factor $\alpha$ of each of the four tables.

(c) Go back to your **linear probing** table. Remove 93 by marking its slot as *deleted*, not empty. To mark a slot as deleted, write **DEL** in it. A DEL slot holds no key, but it is not empty: it records that a key used to be there. (This is the same idea as the `DELETED` marker you will use in the lab.) On the answer sheet, show the table after removing 93 and again after inserting 41 in part (iii).

  (i) Search for 89. How many probes does the search take?

(ii) Suppose the removal had set 93's slot to *empty* instead. What would the search for 89 report, and why is that wrong?

(iii) Now insert 41. Which slot does it go in? How many slots must you check before you can be sure 41 is not already in the table, and why can you not stop at the first deleted slot?

## 5 Problem 2: Find the Mistakes

Jordan inserted these eight keys, in this order, into a table of size $m = 11$, using the same conventions as above:

$$44,\ 39,\ 43,\ 97,\ 38,\ 40,\ 76,\ 10$$

Jordan's work is below. It contains **exactly five mistakes**: one in each of the four tables, and one in the probe counts. For each mistake, say where it is, explain what Jordan did wrong, and give the correct result. Then give the corrected version of each table.

**Table 1:** Jordan's three open-addressing tables. A dash means the slot is empty.

| Slot | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **Linear** | 44 | 76 | – | – | – | 38 | 39 | 40 | – | 97 | 43 |
| **Quadratic** | 44 | 10 | – | 76 | – | 38 | 39 | 40 | – | 97 | 43 |
| **Double hashing** | 44 | 76 | 10 | – | – | 38 | 39 | 40 | – | 97 | 43 |

Under the linear probing table, Jordan wrote: "10 could not be placed, because its home slot is the last slot in the table and it is full."

**Table 2:** Jordan's separate chaining table. Chains that are not listed are empty.

| Chain | Keys, first to last |
|---|---|
| 0 | 44 |
| 5 | 38 |
| 6 | 39 |
| 7 | 40 |
| 8 | 97 |
| 10 | 43, 76, 10 |

**Table 3:** Jordan's probe counts for successful searches in the linear probing table.

| Key searched for | Probes |
|---|---|
| 44 | 1 |
| 43 | 1 |
| 76 | 2 |
| 38 | 1 |

## 6 Problem 3: Predict and Justify

For each part, make your prediction first and explain your reasoning. You may then check it by working it out.

(a) The keys are the 30 multiples of 10 from 10 to 300 (10, 20, 30, ..., 300). How many **different home slots** $h(x)$ do these keys use in a table of size 40? In a table of size 41? Which table would you rather use, and why?

(b) In a table of size 40, quadratic probing from home slot 16 checks slots $(16 + i^2)$ mod 40. List $i^2$ mod 40 for $i = 0$ through 12. How many different slots can a search from slot 16 ever reach? How many could it reach in a table of size 41, and what does that mean for a table that is more than half full?

(c) A school gives every student a five-digit ID, and every student admitted this year has an ID starting with 26. A programmer hashes IDs into a table of size 101 using only the first two digits of the ID. Predict what happens, and propose a better hash function. Which property of a good hash function does the programmer's version break?

## 7 Problem 4: Use the Formulas Backward

Use the efficiency formulas from the notes. Show your work, and round load factors to four decimal places.

(a) You expect to store 600 keys with **linear probing**, and you want an unsuccessful search to take **3 probes or fewer** on average. What is the largest load factor you can allow? What is the smallest **prime** table size that meets the goal?

(b) Repeat (a) for **double hashing**. Which technique needs less memory for the same goal, and by how many slots?

(c) You store the same 600 keys with **separate chaining** in a table of size 211. Give $\alpha$ and the expected probes for a successful and an unsuccessful search. Explain why $\alpha$ can be greater than 1 here but not in (a) or (b).

(d) Your linear probing table from Problem 1 has size 13. A program grows a table when $\alpha$ goes above 2/3, and it grows to the smallest prime that is at least twice the old size. Does the Problem 1 table need to grow? If so, what is the new size and the new $\alpha$? Explain why every key must be inserted again with the new $h(x)$ rather than copied to the same slot number, using key 51 as an example.

## 8 What to Submit

- **The answer sheet**, saved as .docx or .odt, with all four problems.
- The file in the `taskw14` folder of your GitHub repository.